

Custom CMS Multi-Website Project Plan (Next.js + Express)

1. Introduction

This document explains a complete step-by-step theoretical plan to build a custom CMS system that supports multiple websites using Next.js and Express. The goal is to provide a scalable, flexible, and maintainable architecture.

2. Project Goals

The main goal is to create a system where multiple websites can be managed from a single CMS dashboard. Each website will have its own content, design configuration, and settings.

3. Technology Stack Overview

Frontend: Next.js for fast rendering and SEO optimization. Backend: Express.js for API handling and business logic. Database: Any relational or NoSQL database depending on scalability needs.

4. System Architecture

The system will follow a modular architecture. The CMS dashboard will interact with backend APIs, and websites will fetch content dynamically from the CMS.

5. Multi-Website Concept

Each website will be treated as a separate entity with its own configuration, themes, and content, but managed under one unified CMS.

6. Content Management Flow

Admins create pages, manage content, upload media, and configure settings through the dashboard. The backend stores and serves this data to frontend websites.

7. User Roles & Permissions

Different roles like Admin, Editor, and Viewer will control access levels. Permissions ensure security and proper workflow management.

8. Database Design

Core entities include Users, Websites, Pages, Content Blocks, Media, and Settings. Relationships should be designed to support scalability.

9. API Design Strategy

REST or GraphQL APIs will be used to fetch and manage content. APIs should be secure, versioned, and optimized for performance.

10. Development Phases

Phase 1: Planning and requirement gathering. Phase 2: Database and backend setup. Phase 3: CMS dashboard development. Phase 4: Frontend website integration. Phase 5: Testing and optimization.

11. Deployment Strategy

Deploy backend and frontend separately. Use cloud services for scalability. Implement CI/CD pipelines for smooth updates.

12. Performance & Security

Implement caching, authentication, rate limiting, and data validation to ensure system reliability and security.

13. Maintenance & Scaling

Regular updates, monitoring, and scaling strategies should be implemented to handle increasing traffic and new features.

14. Conclusion

This project provides a complete roadmap to build a professional multi-website CMS system using modern technologies like Next.js and Express.